

[Day 1 종합실습] 실행결과 및 코드 분석 의견

SKALA 데이터분석 및 AIOps | 데이터 수집 미니 파이프라인 | 광주캠퍼스 4반 박영서 | 2026-07-20

실행결과 요약

- 실행환경: Python 3.13.3 / venv + requirements.txt로 패키지 관리 (httpx, pydantic, pandas, pyarrow, pytest, ruff)
- Open-Meteo · Countries.dev · ip-api 3개 API를 **asyncio.gather()**로 **동시 수집 — 3/3건 성공**. 실제 16.80초 (순차 합산 추정 18.86초)
- ip-api는 현재 네트워크에서 차단되어 3회 재시도가 모두 타임아웃 → 미러(ipwho.is)로 자동 전환해 수집 성공
- Pydantic v2로 weather 72건, country 1건, ip 1건 검증. 오류 0건
- CSV · Parquet 두 형식으로 저장하고 쓰기/읽기 시간과 파일 크기 비교. 저장 후 재로딩해 72건 일치 확인
- 72행으로는 포맷 차이가 드러나지 않아, 같은 데이터를 500배(36,000행) 복제해 동일 조건으로 재측정함
- 실제 응답이 전부 정상이라 검증 실패 경로가 실행되지 않아, 파이프라인 [5]단계에서 잘못된 값을 주입해 3건 중 2건이 차단되는 것을 확인함
- pytest 16건 전부 통과, ruff check · ruff format 모두 통과

성능 측정 결과

두 포맷에 동일한 반복 횟수(20회)를 적용해 평균을 냈습니다.

구간	format	write_ms	read_ms	size_kb
수집 데이터 (72행)	CSV	0.744	0.529	5.52
	Parquet	0.944	0.966	6.05
확대 데이터 (36,000행)	CSV	80.093	26.189	2,710.06
	Parquet	11.306	2.483	10.79

36,000행 기준 Parquet이 읽기 10.5배 · 쓰기 7.1배 빠르고, 파일 크기는 251.2배 작음

분석 결과에 대한 본인 의견

Parquet이 항상 빠른 건 아니었다 — 손익분기점이 있다

72행 구간에서는 오히려 CSV의 읽기·쓰기가 모두 빨랐습니다. Parquet은 파일 안에 스키마와 컬럼 청크 메타데이터를 함께 저장하는 구조라, 데이터가 적으면 이 고정 비용이 이득을 상회합니다. 반면 36,000행에서는 읽기 10.5배 차이로 완전히 역전됐습니다. “Parquet이 CSV보다 빠르다”는 명제는 대용량을 전제한 말이고, 실제로는 **둘의 우위가 뒤바뀌는 손익분기점이 존재한다**는 것을 실측으로 확인했습니다.

이상한 측정값은 대상보다 측정 방법을 먼저 의심해야 했다

처음 측정했을 때 72행 Parquet 읽기가 184ms로 나왔습니다. 같은 데이터의 CSV 읽기가 0.5ms인데 300배 이상 차이가 나는 건 납득이 안 돼서 확인해보니, pyarrow 엔진을 처음 불러오는 비용이 timeit의 첫 반복 안에 섞여 들어간 것이었습니

다. 측정 전에 쓰기·읽기를 각 1회씩 워밍업하도록 고치자 1ms 아래로 떨어졌습니다. **측정값이 상식과 어긋날 때 결론부터 쓰면 틀린 해석이 남는다는 걸 배웠습니다.**

예외 처리는 "넣었다"가 아니라 "동작한다"를 보여야 한다

채점 기준에 "타입 오류 시 예외 처리 포함"이 있는데, 실제 API 응답이 모두 정상이라 검증 실패 분기가 한 번도 실행되지 않았습니다. 그러면 예외 처리를 작성했다는 사실만 남고 동작한다는 근거는 남지 않습니다. 그래서 파이프라인 마지막 단계에서 범위 초과(강수확률 150%)와 타입 오류(기온 "이십도")를 일부러 주입해, 3건 중 2건이 사유와 함께 차단되는 것을 출력하도록 했습니다. 이 시연은 저장 파일에는 영향을 주지 않습니다.

재시도와 폴백은 성격이 다른 장치다

ip-api가 계속 실패해서 먼저 지수 백오프 재시도(0.5초 → 1.0초, 최대 3회)를 붙였는데, 3회 모두 똑같이 타임아웃했습니다. 원인을 보니 해당 도메인의 80·443 포트가 네트워크 단에서 아예 막혀 있었습니다.

재시도는 일시적 오류(순간적인 네트워크 장애, 5xx)를 전제한 장치라 지속적 차단에는 몇 번을 다시 걸어도 결과가 같습니다. 그래서 재시도(같은 주소, 간격을 늘려가며) 위에 미리 전환(다른 주소)을 한 겹 더 얹었고, 그 결과 3/3 수집에 성공했습니다. **실패 대응은 한 종류가 아니라, 실패의 성격에 따라 총을 나눠야 한다**는 점이 이번 실습의 가장 큰 수확이었습니다.

공급자가 늘면 스키마가 아니라 정규화 계층을 늘렸다

미러(ipwho.is)는 같은 정보를 다른 키 이름으로 줍니다(query↔ip, lat↔latitude, isp↔connection.isp). 공급자별로 Pydantic 모델을 하나씩 만들면 하류 코드도 분기가 생겨서, 대신 `normalize_ip_payload()`에서 한 형태로 맞춘 뒤 기존 `IpInfo` 하나로 검증하게 했습니다. 이렇게 하면 공급자가 더 늘어도 바뀌는 곳은 정규화 함수 한 군데입니다.

수집 실패를 검증 실패로 두 번 세지 않게 했다

수집에 실패한 API를 그대로 검증 단계로 넘기면 같은 사고 하나가 "수집 실패"와 "필드 누락"으로 두 번 기록됩니다. `build_records()`의 `payload_of()`에서 수집 성공분만 통과시키고 실패는 그 자리에서 오류로 남기도록 해, 오류 리포트의 건수가 실제 사고 건수와 일치하게 했습니다.

추가 의견 (개선 사항, 코드 품질 개선 측면)

- 테스트에서 `httpx.MockTransport`로 실패 응답을 흉내 내, 네트워크 없이도 재시도·미러 폴백 경로를 검증했습니다. 외부 상태에 의존하는 코드일수록 테스트에서 그 의존을 끊어내야 한다고 생각합니다
- 예외 처리 시연은 weather 모델로만 합니다. country-ip 모델도 같은 방식으로 확인하면 검증 근거가 더 완전해질 것입니다
- `pytest --cov=src`로 커버리지를 수치화해, 테스트가 닿지 않는 분기를 남은 위험으로 관리하면 좋겠습니다
- 매 실행마다 전체를 새로 받습니다. 마지막 수집 시각을 기록해 변경분만 가져오는 증분 수집으로 바꾸면 API 호출량을 줄일 수 있습니다
- API 응답 구조가 바뀌면 검증이 일괄 실패합니다. 스키마에 버전을 부여하고 검증 실패율이 임계치를 넘으면 알림을 보내는 장치가 필요합니다
- 수집 결과에 `source`(실제 응답을 받아낸 주소)와 `attempts`(시도 횟수)를 남겨, 나중에 로그만 보고도 미러 사용 여부와 재시도 횟수를 판별할 수 있게 했습니다

실행결과

1. python -m src.pipeline — 수집 → 검증 → 저장/성능비교 → 재로딩 검증 → 예외 처리 시연

```
=====
[1] 비동기 수집 - asyncio.gather() 로 3개 API 동시 호출
=====
18:31:46|INFO|httpx|HTTP Request: GET https://countries.dev/alpha/KOR "HTTP/1.1 200 OK"
18:31:46|INFO|src.collect|[country] 수집 성공 (0.93s, 1회 시도)
18:31:46|INFO|httpx|HTTP Request: GET https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.978&hourly=temperature_2m,precipitation_probability&forecast_days=3&timezone=Asia/Seoul "HTTP/1.1 200 OK"
18:31:46|INFO|src.collect|[weather] 수집 성공 (1.18s, 1회 시도)
18:31:50|WARNING|src.collect|[ip] 1/3회차 실패(timeout(5.0s)) - 0.5s 후 재시도
18:31:55|WARNING|src.collect|[ip] 2/3회차 실패(timeout(5.0s)) - 1.0s 후 재시도
18:32:01|WARNING|src.collect|[ip] 1차 주소 실패(timeout(5.0s)) - 미러로 전환: https://ipwho.is/8.8.8.8
18:32:01|INFO|httpx|HTTP Request: GET https://ipwho.is/8.8.8.8 "HTTP/1.1 200 OK"
18:32:01|INFO|src.collect|[ip] 수집 성공 (16.74s, 4회 시도) (미러 사용)
18:32:01|INFO|src.collect|동시 수집 완료: 3/3건 성공 | 실제 16.80s (순차 합산 추정 18.86s)
18:32:01|INFO|src.collect|원본 응답 저장: api_responses.json
    weather : 성공 (1.18s)
    country : 성공 (0.93s)
    ip      : 성공 (16.74s)

=====
[2] 스키마 검증 - Pydantic v2 로 타입·범위 확인
=====
18:32:01|INFO|src.transform|[weather] 검증 결과: 유효 72건 / 오류 0건
18:32:01|INFO|src.transform|[country] 검증 성공: Korea (Republic of) / 수도 Seoul
18:32:01|INFO|src.transform|[ip] 검증 성공: 8.8.8.8 → United States/San Jose
18:32:01|INFO|src.transform|저장 대상 레코드 72건 / 누적 오류 0건
    검증 통과 72건 / 오류 0건
    첫 레코드: {'time': '2026-07-20T00:00', 'temperature_c': 23.2, 'precipitation_probability': 6, 'country': 'Korea (Republic of)', 'capital': 'Seoul', 'population': 51780579, 'observer_ip': '8.8.8.8', 'observer_city': 'San Jose'}
18:32:01|INFO|src.storage|오류 리포트 저장: errors.json (0건)

=====
[3] 저장 및 성능 비교 - CSV vs Parquet
=====
데이터 형태: 72행 × 8열
컬럼: ['time', 'temperature_c', 'precipitation_probability', 'country', 'capital', 'population', 'observer_ip', 'observer_city']

● 수집 데이터 (72행) - 반복 20회 평균
18:32:01|INFO|src.storage|[CSV] 저장 완료: weather_seoul.csv
18:32:02|INFO|src.storage|[Parquet] 저장 완료: weather_seoul.parquet
format write_ms read_ms size_kb
CSV      0.744    0.529    5.52
Parquet   0.944    0.966    6.05

● 확대 데이터 (36,000행) - 동일 조건 재측정
18:32:04|INFO|src.storage|[CSV] 저장 완료: weather_scaled.csv
18:32:04|INFO|src.storage|[Parquet] 저장 완료: weather_scaled.parquet
format write_ms read_ms size_kb
CSV      80.093   26.189   2710.06
Parquet  11.306    2.483    10.79

→ 확대 데이터(36,000행) 기준 Parquet 우위: 읽기 10.5배 빠름, 쓰기 7.1배 빠름, 파일 크기 251.2배 작음
(72행 소량 구간에서는 CSV가 근소하게 빨랐다 - Parquet의 이점은 데이터가 커질수록 드러난다)

=====
[4] 저장 결과 재로딩 검증
=====
18:32:04|INFO|src.storage|재로딩 검증 통과: CSV·Parquet 모두 72건
CSV·Parquet 모두 72건으로 재로딩 확인
```

```
=====
[5] 예외 처리 시연 - 잘못된 값이 실제로 걸러지는지 확인
=====
```

```
18:32:04|WARNING|src.transform|[weather] 2행 검증 실패 - precipitation_probability: Input should be less than or equal to 100
```

```
18:32:04|WARNING|src.transform|[weather] 3행 검증 실패 - temperature_c: Input should be a valid number, unable to parse string as a number
```

```
18:32:04|INFO|src.transform|[weather] 검증 결과: 유효 1건 / 오류 2건
```

```
주입 3건 → 통과 1건 / 차단 2건
```

```
[차단] 2행 → precipitation_probability: Input should be less than or equal to 100
```

```
[차단] 3행 → temperature_c: Input should be a valid number, unable to parse string as a number
```

```
[OK] 파이프라인 정상 완료
```

3개 API 동시 수집(3/3 성공, ip는 재시도 3회 후 미러 전환), Pydantic 검증 72건 전부 통과, CSV·Parquet 저장 및 두 구간 성능 비교, 재로딩 72건 일치, 오류 주입 시 3건 중 2건 차단 확인

2. pytest -v — 검증·재시도·폴백 테스트

```
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0 -- /Users/youngseo/Desktop/skala/광주캠퍼스_4반_박영
서_day1종합실습/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/youngseo/Desktop/skala/광주캠퍼스_4반_박영서_day1종합실습
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collecting ... collected 16 items

tests/test_pipeline.py::test_weather_record_accepts_valid_row PASSED [ 6%]
tests/test_pipeline.py::test_weather_record_rejects_invalid[precipitation_probability-150] PASSED [ 12%]
tests/test_pipeline.py::test_weather_record_rejects_invalid[temperature_c-999] PASSED [ 18%]
tests/test_pipeline.py::test_weather_record_rejects_invalid[time-2026-07-20] PASSED [ 25%]
tests/test_pipeline.py::test_country_requires_alpha3_length PASSED [ 31%]
tests/test_pipeline.py::test_ip_info_rejects_out_of_range_latitude PASSED [ 37%]
tests/test_pipeline.py::test_extract_weather_rows_zips_columns PASSED [ 43%]
tests/test_pipeline.py::test_validate_weather_splits_valid_and_errors PASSED [ 50%]
tests/test_pipeline.py::test_build_records_attaches_context PASSED [ 56%]
tests/test_pipeline.py::test_build_records_reports_failed_api PASSED [ 62%]
tests/test_pipeline.py::test_to_dataframe_raises_on_empty PASSED [ 68%]
tests/test_pipeline.py::test_to_dataframe_builds_expected_shape PASSED [ 75%]
tests/test_pipeline.py::test_fetch_with_retry_succeeds_after_failures PASSED [ 81%]
tests/test_pipeline.py::test_fetch_with_retry_gives_up_after_max_attempts PASSED [ 87%]
tests/test_pipeline.py::test_fetch_falls_back_to_mirror PASSED [ 93%]
tests/test_pipeline.py::test_normalize_ip_payload_unifies_providers PASSED [100%]

===== 16 passed in 0.57s =====
```

스키마 검증 12건 + 재시도·미러 폴백·응답 정규화 4건, 총 16건 전부 PASSED

3. ruff check . / ruff format --check .

```
$ ruff check .
All checks passed!

$ ruff format --check .
9 files already formatted
```

코드 스타일 검사 All checks passed, 9개 파일 포맷 준수

채점 기준 대응

항목	대응 내용
환경 구성 + 비동기 수집 (35점)	venv + requirements.txt로 설치, <code>asyncio.gather()</code> 로 3개 API 동시 수집 3/3건 성공 확인
스키마 검증 + 저장 비교 (45점)	Pydantic v2 모델 3종 정의, 타입·범위 오류를 <code>ValidationError</code> 로 분리 기록하고 실행 화면에서 차단 동작 시연, CSV·Parquet 저장 후 쓰기/읽기 시간·크기 측정 출력
테스트 · 커밋 (10점)	pytest 16건 통과, ruff check 오류 없음, Git 커밋 이력 존재 (GitHub 공개 저장소)
완성도 (10점)	모듈·함수 단위 docstring과 의도 주석 작성, 진입점에 프로그램 개요·모듈 구성·변경 내역 명시